

system-testing

May 16, 2026

1 System Testing

1.1 System Overview

The system generates AHP weights from stakeholder preference ratings and then uses those weights as the input for the TOPSIS ranking process. Stakeholders do not provide traditional AHP pairwise comparisons by answering questions such as, “How much more important is criterion X than criterion Y?” Instead, the system asks stakeholders to rate each criterion independently, using linguistic values such as Very Low, Low, Medium, High, and Very High. These linguistic ratings are converted into numeric values, and the pairwise comparison matrix is then derived by comparing each criterion rating against every other criterion rating. For example, if a stakeholder rates Expected Crime Reduction as 5 and Implementation Cost as 3, the generated pairwise value is calculated as $5 / 3$. This creates a reciprocal pairwise matrix that can be processed by AHP, but it is important to note that the matrix is indirectly generated from independent criterion ratings rather than directly collected pairwise judgments.

After individual stakeholder matrices are generated, the system aggregates preferences by stakeholder type and applies stakeholder voting power when producing the final group pairwise matrix. AHP is then used to calculate the final criteria weights. These weights are passed into TOPSIS, where the scenario decision matrix is normalized, multiplied by the AHP-derived weights, and compared against the positive ideal and negative ideal solutions. The system calculates each alternative’s distance from the ideal and negative ideal solutions, computes the closeness coefficient, and ranks alternatives based on those coefficients.

1.2 System Limitations

This system’s process has several limitations that may affect results. Because the pairwise matrix is derived from independent ratings instead of direct criterion-to-criterion comparisons, the AHP weights may not fully represent the stakeholder’s actual trade-off reasoning. The linguistic scale also assumes that values such as Low, Medium, and High can be treated as evenly spaced numeric values, which may oversimplify stakeholder intent. In addition, the generated pairwise matrix may appear mathematically consistent because it is created from rating ratios, even though the stakeholder was not asked to make explicit pairwise judgments. Therefore, manual validation should compare results against the system’s derived-rating method, not against a traditional AHP survey process. These limitations should be considered when interpreting stability tests, rank reversals, and differences between system-generated, manually calculated, and coded validation results.

1.3 Testing Overview

Now that the AHP-to-TOPSIS pipeline has been completed validation testing is required to confirm that the system produces accurate results. With the dashboard in place stakeholder preferences can be captured and processed as intended. This testing will compare system-generated results with manually calculated results to verify that the pipeline functions correctly and produces consistent outputs.

1.4 Testing Scenario

The testing scenario and its data were created manually and are not based on any known policy scenarios.

1.4.1 Scenario Description

A generic city is considering how to allocate a constrained public safety budget. Four distinct alternatives represent different strategic approaches: building new infrastructure, expanding existing capacity, increasing visibility through patrols, or investing in community-based prevention. This scenario brings together diverse stakeholders—from police leadership and city government to community advocates and residents—to weigh the trade-offs between cost, crime reduction effectiveness, community trust impact, and implementation speed. The decision involves balancing public safety improvements against resource constraints and community preferences for different intervention strategies.

1.4.2 Alternatives

- Build new precinct
- Expand existing precinct
- Increase patrol units
- Invest in community violence prevention

1.4.3 Criteria

- Implementation Cost
- Expected Crime Reduction
- Community Trust Impact
- Implementation Time

1.4.4 Stakeholders

- Community Residents (0.15)
- Police Leadership (0.20)
- City Government (0.25)
- Community Advocates & Social Services (0.25)
- Business & Economic Development (0.15)

1.4.5 Data

alternative	cost	crime_reduction	community_trust	months_to_implement
new_precinct	2500000	10.7	89	30
expand_precinct	750000	3.8	91	21
more_patrols	250000	2.5	71	6
community_prevention	220000	2.6	68	7

```
[1]: import pandas as pd
import numpy as np

from pyDecision.algorithm import topsis_method, ahp_method
from IPython.display import display

def load_data(file_path):
    try:
        return pd.read_csv(file_path)
    except Exception as e:
        print(f"Error loading data: {e}")
        return None

def _convert_preference(pref):
    if pref == "Very Low":
        return 1
    elif pref == "Low":
        return 2
    elif pref == "Medium":
        return 3
    elif pref == "High":
        return 4
    elif pref == "Very High":
        return 5
    else:
        return None

def convert_linguistic_to_numeric(df):
    df["implementation_cost"] = df["implementation_cost"].
    ↪apply(_convert_preference)
    df["expected_crime_reduction"] = df["expected_crime_reduction"].
    ↪apply(_convert_preference)
    df["community_trust_impact"] = df["community_trust_impact"].
    ↪apply(_convert_preference)
    df["implementation_time"] = df["implementation_time"].
    ↪apply(_convert_preference)
    return df
```

```

def create_individual_pairwise_matrix(df):
    """
    Creates pairwise comparison matrices grouped by stakeholder_type.

    For each stakeholder group:
    - Builds individual 4x4 pairwise matrices for each stakeholder
    - Aggregates multiple stakeholders in the same group using geometric mean

    Returns a dict mapping stakeholder_type to aggregated 4x4 pairwise matrix.

    Args:
        df: DataFrame with columns [stakeholder_type, implementation_cost,
        ↪ expected_crime_reduction,
        community_trust_impact, implementation_time]

    Returns:
        dict: {stakeholder_type: aggregated_pairwise_matrix_4x4}
    """
    criteria_cols = ["implementation_cost", "expected_crime_reduction",
    ↪ "community_trust_impact", "implementation_time"]

    # Group by stakeholder_type
    grouped = df.groupby("stakeholder_type")

    result = {}

    for stakeholder_type, group_df in grouped:
        # Extract numeric values for this stakeholder group
        group_data = group_df[criteria_cols].to_numpy() # shape:
        ↪ (n_individuals, 4)

        # Build individual pairwise matrices for each stakeholder in the group
        individual_matrices = []

        for row in group_data:
            # Create 4x4 pairwise matrix: matrix[i][j] = row[i] / row[j]
            pairwise_matrix = np.zeros((4, 4))
            for i in range(4):
                for j in range(4):
                    pairwise_matrix[i][j] = float(row[i] / row[j])
            individual_matrices.append(pairwise_matrix)

        # Aggregate using geometric mean
        if len(individual_matrices) == 1:
            # Single stakeholder in group: use as-is
            aggregated_matrix = individual_matrices[0]

```

```

        else:
            # Multiple stakeholders: compute geometric mean element-wise
            # Geometric mean: (product) ^ (1/n)
            aggregated_matrix = np.ones((4, 4))
            for i in range(4):
                for j in range(4):
                    product = 1.0
                    for individual_matrix in individual_matrices:
                        product *= individual_matrix[i][j]
                    aggregated_matrix[i][j] = product ** (1.0 /
↪len(individual_matrices))

            result[stakeholder_type] = aggregated_matrix

    return result

def create_aggregate_pairwise_matrix(grouped_matrices, stakeholder_weights):
    """
    Aggregates pairwise matrices across stakeholder groups using weighted
↪geometric mean.

    Args:
        grouped_matrices: dict mapping stakeholder_type to 4x4 pairwise matrix
        stakeholder_weights: dict mapping stakeholder_type to voting_power (0-1)

    Returns:
        numpy array: 4x4 aggregated pairwise matrix
    """
    # Validate that weights sum to 1 (approximately)
    weights_sum = sum(stakeholder_weights.values())
    if weights_sum == 0:
        raise ValueError("Stakeholder weights sum to 0")

    # Renormalize weights to sum to 1
    normalized_weights = {k: v / weights_sum for k, v in stakeholder_weights.
↪items()}

    # Initialize aggregated matrix with ones
    aggregated_matrix = np.ones((4, 4))

    # For each matrix position, compute weighted geometric mean
    for i in range(4):
        for j in range(4):
            product = 1.0
            for stakeholder_type, matrix in grouped_matrices.items():
                weight = normalized_weights.get(stakeholder_type, 0)

```

```

        if weight > 0:
            # Use weight as exponent in geometric mean formula
            product *= matrix[i][j] ** weight
        aggregated_matrix[i][j] = product

    return aggregated_matrix

if __name__ == "__main__":
    data = load_data("./system-testing-assets/data.csv")

    if data is None:
        raise ValueError("Failed to load required data file.")

    criteria = ["cost", "crime_reduction", "community_trust", "months_to_implement"]

    criteria_types = ["min", "max", "max", "min"]

    display(data)

```

	alternative	cost	crime_reduction	community_trust \
0	new_precinct	2500000	10.7	89
1	expand_precinct	750000	3.8	91
2	more_patrols	250000	2.5	71
3	community_prevention	220000	2.6	68

	months_to_implement
0	30
1	21
2	6
3	7

1.5 Trials

The system will be tested by conducting multiple trials and comparing the system-generated results with results obtained through manual calculations and coded validation. The tests will be divided into three groups, each with a different number of participants, and each group will then undergo stability testing.

1.5.1 Rating Mapping

Linguistic	Numeric
Very Low	1
Low	2
Medium	3
High	4
Very High	5

1.5.2 First Trial

The first trial will be a balanced pilot trial with one voice from each stakeholder group. This is best for manual verification because each stakeholder perspective is easy to trace.

Participants

Participant	Stakeholder Type	Implementation Cost	Expected Crime Reduction	Community Trust Impact	Implementation Time	Story / Comment
Jordan Reed	community_residents	High	Medium	Very High	Medium	Resident voice prioritizes community trust while still supporting practical safety improvements.
Maya Chen	business_economy	Medium	Very High	Low	Very High	Business owner wants rapid visible crime reduction to protect storefronts and customers.
Sofia Alvarez	police_leaders	Low	Very High	Medium	High	Public safety representative prioritizes crime reduction and operational speed over cost control.

Participant	Stakeholder Type	Implementation Cost	Expected Crime Reduction	Community Trust Impact	Implementation Time	Story / Comment
Ethan Brooks	city_government	High	High	High	Medium	Planner prefers a balanced policy that manages cost, safety impact, and public legitimacy.
Ari Morgan	community_advocates	Medium	Low	Very High	Low	Civil rights advocate is cautious about rapid enforcement-heavy responses and emphasizes trust.

Stakeholder Grouped Pairwise Matrices

```
[13]: trial1_test1_raw = {
    "participant": ["Jordan Reed", "Maya Chen", "Sofia Alvarez", "Ethan Brooks", "Ari Morgan"],
    "stakeholder_type": ["community_residents", "local_business_owners", "public_safety_officials", "city_planners", "civil_rights_advocates"],
    "implementation_cost": ["High", "Medium", "Low", "High", "Medium"],
    "expected_crime_reduction": ["Medium", "Very High", "Very High", "High", "Low"],
    "community_trust_impact": ["Very High", "Low", "Medium", "High", "Very High"],
    "implementation_time": ["Medium", "Very High", "High", "Medium", "Low"]
}

df = pd.DataFrame(trial1_test1_raw)
df = convert_linguistic_to_numeric(df)

# Create individual pairwise matrices grouped by stakeholder_type
grouped_matrices = create_individual_pairwise_matrix(df)

print("\n=== Grouped Pairwise Matrices by Stakeholder Type ===")
for stakeholder_type, matrix in grouped_matrices.items():
    print(f"\n{stakeholder_type}:")
```



```
print(matrix)
```

=== Grouped Pairwise Matrices by Stakeholder Type ===

```
city_planners:
[[1.          1.          1.          1.33333333]
 [1.          1.          1.          1.33333333]
 [1.          1.          1.          1.33333333]
 [0.75        0.75        0.75        1.          ]]
```

```
civil_rights_advocates:
[[1.          1.5         0.6         1.5         ]
 [0.66666667  1.          0.4         1.          ]
 [1.66666667  2.5         1.          2.5         ]
 [0.66666667  1.          0.4         1.          ]]
```

```
community_residents:
[[1.          1.33333333  0.8         1.33333333]
 [0.75        1.          0.6         1.          ]
 [1.25        1.66666667  1.          1.66666667]
 [0.75        1.          0.6         1.          ]]
```

```
local_business_owners:
[[1.          0.6         1.5         0.6         ]
 [1.66666667  1.          2.5         1.          ]
 [0.66666667  0.4         1.          0.4         ]
 [1.66666667  1.          2.5         1.          ]]
```

```
public_safety_officials:
[[1.          0.4         0.66666667  0.5         ]
 [2.5         1.          1.66666667  1.25        ]
 [1.5         0.6         1.          0.75        ]
 [2.          0.8         1.33333333  1.          ]]
```

AHP Aggregate Weight Generation

```
[14]: # Define stakeholder voting power (from scenario.json)
stakeholder_weights = {
    "community_residents": 0.15, # Residents
    "local_business_owners": 0.15, # Business & Economic Development
    "public_safety_officials": 0.20, # Police Leadership
    "city_planners": 0.25, # City Government
    "civil_rights_advocates": 0.25 # Community Advocates & Social Services
}

# Create aggregate pairwise matrix weighted by stakeholder voting power
```

```

final_pairwise_matrix = create_aggregate_pairwise_matrix(grouped_matrices,
↳ stakeholder_weights)

print("\n=== Final Aggregated Pairwise Matrix ===")
print(final_pairwise_matrix)

# Compute AHP weights from aggregated pairwise matrix
ahp_weights, rc = ahp_method(final_pairwise_matrix, wd="mean")

print("\n=== AHP Weights ===")
for i in range(0, ahp_weights.shape[0]):
    print(f"{criteria[i]}: {ahp_weights[i]:.4f}")

print(f"\nConsistency Ratio (RC): {rc:.4f}")

```

```

=== Final Aggregated Pairwise Matrix ===
[[1.          0.89104228 0.83405895 1.00118653]
 [1.1222812  1.          0.93604868 1.12361282]
 [1.19895602 1.0683205  1.          1.20037862]
 [0.99881488 0.88998628 0.83307049 1.          ]]

```

```

=== AHP Weights ===
cost: 0.2315
crime_reduction: 0.2598
community_trust: 0.2775
months_to_implement: 0.2312

```

Consistency Ratio (RC): 0.0000

Stakeholder Dashboard Results

Group Aggregate Pairwise Matrix

The power-weighted geometric mean of all stakeholder pairwise matrices, used to derive the group AHP criterion weights above.

	implementation_cost	expected_crime_reduction	community_trust_impact	implementation_time
implementation_cost	1	0.891	0.8341	1.0012
expected_crime_reduction	1.1223	1	0.936	1.1236
community_trust_impact	1.199	1.0683	1	1.2004
implementation_time	0.9988	0.89	0.8331	1

Group Weighting Summary

Criterion	Weight
community_trust_impact	0.2775
expected_crime_reduction	0.2598
implementation_cost	0.2315
implementation_time	0.2312

Consistency Ratio0.0

Consistent?Yes

Aggregationweighted_geometric...

Results Based on the results of the test we can see that we have identical results.